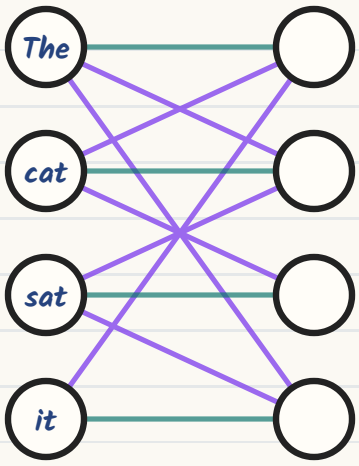
TRANSFORMERS

COMPLETE NOTES

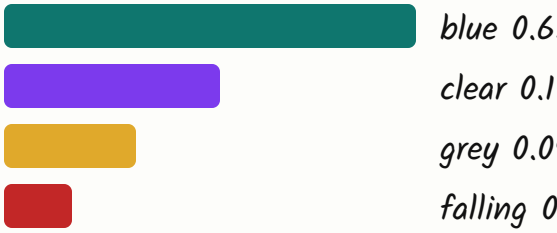
≋ (How an LLM Actually Works) ≋

ATTENTION



PREDICT NEXT TOKEN

"The sky is ____"



one word at a time, again & again!

BEGINNER TO ADVANCED

SWIPE TO NEXT →

🔤 TOKENS & EMBEDDINGS

🎯 Q, K, V & MULTI-HEAD

🏋️ TRAINING & FINE-TUNING

⚡ KV CACHE & DECODING

20 pages · plain English · interview ready ✍️

👤➕ FOLLOW | 📤 SHARE | 💬 COMMENT | ❤️ LIKE

What is an LLM? Why Transformers?

The one-line truth

An LLM (Large Language Model) is a giant ***next-word guessing machine***. You give it some text, it gives back the most likely next piece of text. That is all it does. Everything else, chat, code, translation, essays, comes from doing this guess millions of times.

A Transformer is the design of the machine that does the guessing. Introduced in the 2017 Google paper *"Attention Is All You Need"*. Almost every model you have heard of, GPT, Claude, Gemini, Llama, is a Transformer inside.

SIMPLE ANALOGY 🧠

Think of a very well-read person who has read most of the internet, has no memory of you, and can only answer by writing the next word. Give them the whole conversation as a note, they read it, write one word, then read the note again including their own word. Repeat.

Why it beat the old way

- ✓ Parallel: reads all words at once, not one by one → trains fast on GPUs
- ✓ Long memory: word 1 can look straight at word 5000
- ✓ Scales: more data + more layers = reliably better
- ✓ General: same design works for text, code, images, audio

RNN (old) : one word at a time



TRANSFORMER : all at once



Old models vs Transformer

Point	RNN / LSTM (before 2017)	Transformer (2017 →)
How it reads	Word by word, in order	Whole sentence in one shot
Memory	One hidden state, forgets far-back words	Every word can look at every other word
Training speed	Cannot parallelise across time steps	Fully parallel → made huge models possible
Cost weakness	Long chains of gradients (vanishing)	Cost grows with square of text length (n^2)
Used today for	Small on-device tasks, time series	Nearly every modern LLM

Where you meet it daily

- 💬 Chat assistants and customer support bots
- 💻 Code completion (Copilot, Cursor)
- 🌐 Translation and subtitles
- 📄 Summarising docs, RAG search over company data
- 🔎 Semantic search and recommendations

💡 Remember

The model does not "know" facts the way a database does. Facts are baked into its weights during training, which is why it can be confidently wrong.

⭐ Interview Tip

Q: *In one line, what problem did the Transformer solve?* → It removed recurrence, so a sequence model could be trained in parallel and still connect distant words directly, which is what allowed models to scale to billions of parameters.

The Whole Journey : Text In → Text Out

every single word you get back travels this exact path

Step by step

1 Tokenize

Your sentence is chopped into small pieces called tokens, each mapped to an ID number.

2 Embed

Each ID becomes a long list of numbers (a vector) that carries meaning.

3 Add position

Order information is mixed in, otherwise the model cannot tell "dog bites man" from "man bites dog".

4 Attention

Every token looks at the earlier tokens and pulls in whatever is relevant to it.

5 Feed-forward

Each token is then processed on its own through a small neural network that adds stored knowledge.

6 Repeat N times

Steps 4 and 5 form one block. Stack 32, 80, 100+ of them. Early blocks catch grammar, later ones catch meaning.

7 Score every word

The final vector is compared against the whole vocabulary → one raw score (logit) per possible token.

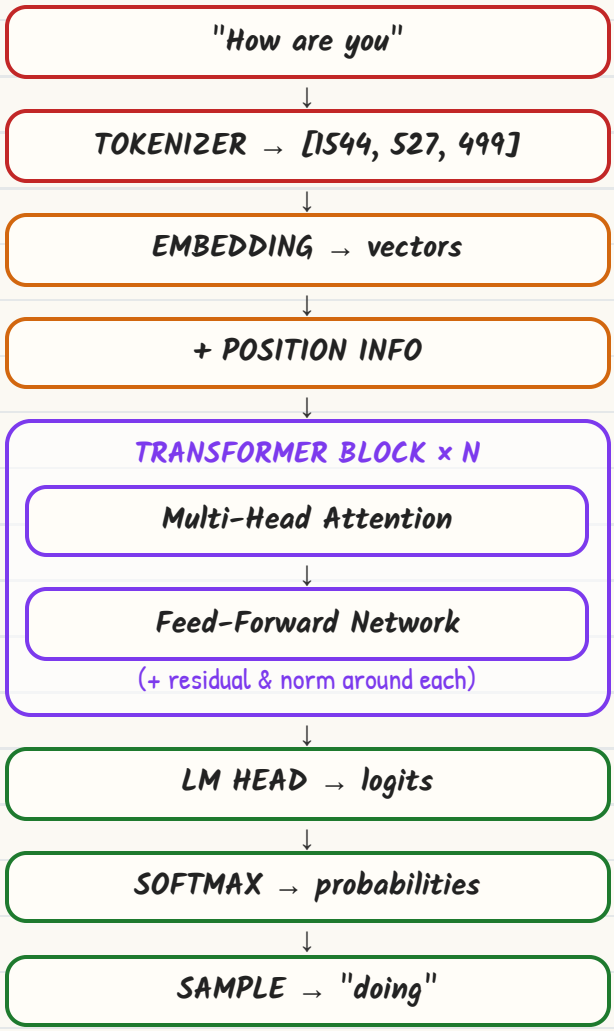
8 Softmax

Scores turn into probabilities that add up to 1.

9 Pick & loop

One token is chosen, glued to the end of your text, and the whole thing runs again for the next token.

The pipeline



↻ then start again!

💡 Key Idea

Attention mixes information between tokens. The feed-forward layer thinks about each token alone. A Transformer is just those two moves, repeated.

⭐ Interview Tip

Be able to recite this pipeline in 30 seconds: tokens → embeddings → +position → (attention + FFN) × N → logits → softmax → sample → repeat.

Step 1 : Tokenization

the model never sees letters or words, only numbers

What is a token?

A token is a chunk of text: sometimes a whole word, sometimes a piece of one, sometimes just a space or punctuation mark. The tokenizer has a fixed **vocabulary** (say 50k to 200k entries) and every token maps to an integer ID.

```
# how text becomes numbers
"Transformers are unbelievable!"
  ↓ split
["Trans", "formers", " are", " un",
 "bel", "iev", "able", "!"]
  ↓ look up in vocab
[41541, 388, 527, 653, 9978, 30963, 481, 0]
```

Common words get their own token. Rare or made-up words get broken into pieces, so the model can still handle anything you type, including typos and new words.

Why not just use words?

- ✗ Whole words: vocabulary would be millions, and any new word breaks it
- ✗ Single letters: sequences get very long and slow, meaning gets thin
- ✓ Sub-words are the sweet spot: small fixed vocabulary + can spell out anything

HOW BPE BUILDS A VOCAB 🔧

Byte Pair Encoding: start with single characters, then repeatedly find the most frequent adjacent pair in the training text and merge it into one new token. Do this ~50,000 times. Frequent patterns like "ing", "tion", " the" naturally become single tokens.

Rules of thumb worth memorising

Thing	Rough number
1 token	≈ 4 characters of English ≈ ¾ of a word
100 tokens	≈ 75 English words ≈ half a page
GPT-2 vocab	50,257 tokens
Llama 3 vocab	128,256 tokens
Other languages	Hindi, Tamil, Chinese often need 2–4× more tokens for the same meaning → costs more
Code	Indentation and symbols eat tokens fast

Popular tokenizers

- BPE / byte-level BPE → GPT family, Llama
- WordPiece → BERT
- SentencePiece / Unigram → T5, many multilingual models

Byte-level means the fallback unit is a raw byte, so the tokenizer can never fail on an emoji or an unknown script.

💡 Why it matters at work

API billing, context limits and latency are all counted in tokens, not words. Trimming a prompt by 30% is a direct 30% cost saving.

Famous tokenizer quirks 🤯

- "How many r's in strawberry?" is hard because the model sees 3 chunks, not 10 letters
- Reversing a string or counting characters is unreliable for the same reason
- Arithmetic is shaky partly because numbers split oddly ("1234" may be "12" + "34")
- A leading space matters: " cat" and "cat" are different tokens

⭐ Interview Tip

Q: *Why do LLMs struggle to count letters in a word?* → Because the smallest unit they perceive is a sub-word token, not a character. The letters inside a token are not separately visible to the model.

Step 2 : Embeddings

turning a token ID into meaning the model can do maths on

The idea in one picture

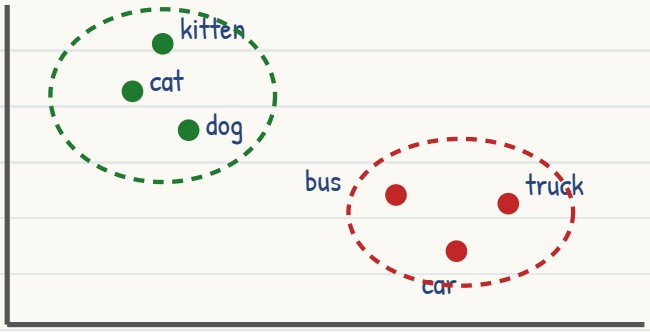
A token ID like 1544 is just a label, it carries no meaning. So the model keeps a giant lookup table called the embedding matrix: one row per vocabulary entry, each row a list of numbers (often 768, 4096 or 12288 long).

```
# embedding lookup = one table row
token "cat"  → id 8415 → row 8415
  = [0.21, -1.04, 0.88, ... , 0.07]
      └── d_model numbers ──┘
```

Each number is a learned **feature**. Nobody programs what they mean, training decides. Some directions end up capturing "is it an animal", "is it plural", "is it formal", and thousands of things with no clean human name.

These vectors are learned during training, exactly like every other weight in the model.

Meaning becomes distance



similar meaning → close together

Words used in similar ways end up with similar vectors. That is what lets the model generalise: it has never seen your exact sentence, but it has seen the neighbourhood.

Classic example: vector("king") − vector("man") + vector("woman") lands near vector("queen"). Relationships live as directions in the space.

Sizes you should recognise

Model	d_model (vector size)	Layers
Original Transformer (base)	512	6 encoder + 6 decoder
BERT-base	768	12
GPT-3 (175B)	12,288	96
Llama-3 8B	4,096	32

Embedding parameters alone = vocab × d_model. For GPT-3 that is over 600 million numbers just to store word meanings.

Two words you will hear

- Static embedding (Word2Vec, GloVe): one fixed vector per word, "bank" always identical
- Contextual embedding (Transformer): the vector changes as it passes through layers, so "river bank" and "bank loan" separate

💡 Real-world link

Vector databases, semantic search and RAG all use these embeddings. Similarity is usually measured with cosine similarity, the angle between two vectors.

⭐ Interview Tip

Q: *What is the difference between an embedding and a one-hot vector?* → One-hot is sparse, huge and has no notion of similarity (every word is equally far from every other). An embedding is dense, small and learned, so distance in the space carries meaning.

Step 3 : Positional Encoding

attention has no sense of order, so we hand it a ruler

The problem

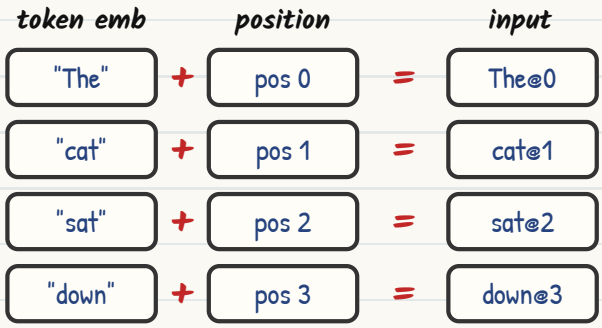
Attention treats the input as a bag of tokens. Shuffle the words and, without position information, the maths gives the same answer. But language is not a bag:

"Dog bites man" ≠ "Man bites dog"
same words, opposite news story 📰

So before the first block, we add a second vector to each embedding that says ***"you are token number 5"***. Same size as the embedding, so it just gets added on top.

```
final_input = token_embedding + position_vector
```

Visual



Now the exact same word carries a different vector depending on where it sits in the sentence.

Three ways to do it

Method	How it works	Where used	Trade-off
Sinusoidal (fixed)	Sine and cosine waves of different frequencies give each position a unique pattern	Original 2017 paper	No parameters, extends past training length in theory
Learned absolute	A trainable vector per position, just like a word embedding	BERT, GPT-2	Simple, but hard limit at max trained position
RoPE (rotary)	Rotates the Q and K vectors by an angle based on position, so attention naturally sees relative distance	Llama, Mistral, most modern LLMs	Best at extending context; today's default

Absolute vs relative, in plain words

- Absolute: "you are word number 7"
- Relative: "you are 3 words to the left of me"

Relative is usually what grammar cares about. "not" changes the meaning of whatever comes just after it, whether that happens at position 4 or position 400. This is why RoPE and relative schemes generalise better to long documents.

Context-extension tricks like RoPE scaling / YaRN stretch these angles so a model trained on 8K tokens can work at 32K or more.

💡 Quick check

If someone claims positional encoding is optional, ask them how attention alone would distinguish "A owes B ₹500" from "B owes A ₹500".

⭐ Interview Tip

Q: *Why sine and cosine?* → Each position gets a unique multi-frequency fingerprint, and the offset between two positions can be written as a linear function of them, so the model can learn relative distance easily.

Inside ONE Transformer Block

the sandwich that gets stacked 32, 80, 96 times

The block (pre-norm style)

input x (one vector per token)

↓

LayerNorm

↓

MULTI-HEAD ATTENTION

tokens talk to each other

↓

⊕ add input back (residual)

↓

LayerNorm

↓

FEED-FORWARD NETWORK

each token thinks alone

↓

⊕ add again (residual)

↓

output → next block

What each piece is for

① LayerNorm

Rescales each token's vector to a steady mean and spread. Keeps numbers from exploding or vanishing as depth grows. Without it, deep stacks simply do not train.

② Attention = communication

The only place where information moves between positions. Each token gathers what it needs from earlier tokens.

③ Residual = memory lane

The original vector is added back, so each sub-layer only has to learn a small correction, not rebuild everything. Gradients also flow straight down this highway during training.

④ FFN = computation

A two-layer network applied to every token separately and identically. This is where most factual knowledge is stored.

THE RESIDUAL STREAM 🛣️

Picture a wide conveyor belt running from input to output. Attention and FFN do not replace what is on the belt, they read it and add to it. Deeper layers therefore refine, not overwrite.

Depth = layers of understanding

Depth	Roughly what it learns
Early blocks	Spelling, word shape, part of speech, local phrases
Middle blocks	Syntax, who-did-what-to-whom, pronoun resolution, entities
Late blocks	Task intent, tone, facts, planning the next token

Pre-norm vs post-norm

- Post-norm (2017 paper): norm after the residual add. Needs warm-up, unstable when deep.
- Pre-norm (modern): norm before each sub-layer. Trains far more stably at 30+ layers, so nearly all LLMs use it.

⭐ Interview Tip

Q: *Which part of the block mixes tokens?* → Only attention. The FFN, LayerNorm and residual all act on each token independently. That single sentence tells an interviewer you actually understand the architecture.

Self-Attention (1) : Query, Key, Value

the heart of the Transformer, explained without maths

The library analogy 📚

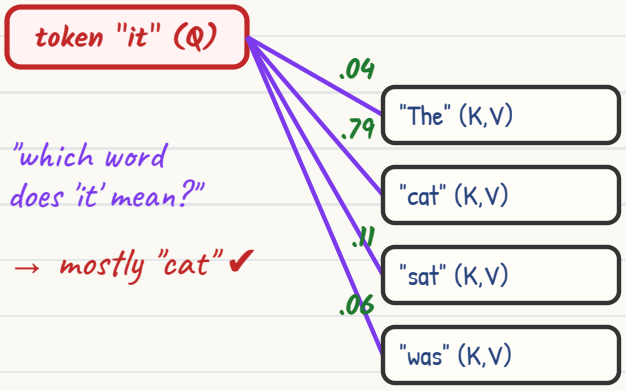
You walk into a library with a question. That question is your Query. Every book has a title on its spine, that is its Key. You compare your question with all the spines, pick the ones that match, and read what is inside those books, the Value.

Attention does exactly this, except every token is simultaneously a reader and a book, and instead of picking one book it takes a **weighted blend** of all of them.

Name	Plain meaning	Made by
Query (Q)	"What am I looking for?"	$x \times W_Q$
Key (K)	"What do I advertise about myself?"	$x \times W_K$
Value (V)	"What information do I actually pass on?"	$x \times W_V$

W_Q, W_K, W_V are learned weight matrices, the same three for every token in that head. Self-attention means Q, K and V all come from the same sentence.

One token's turn



The token "it" ends up carrying most of "cat" inside it. That is how a pronoun gets resolved, with no grammar rules written by anyone.

Three flavours of attention

Type	Q comes from	K, V come from	Used in
Self-attention	the sequence	the same sequence	every Transformer
Masked self-attn	the sequence	only earlier positions	GPT-style decoders
Cross-attention	the decoder	the encoder output	translation, T5, image-text models

💡 Mental model

Attention is a soft, learned lookup table. A database returns the one row whose key matches exactly. Attention returns a mixture of all rows, weighted by how well each key matches the query.

⭐ Interview Tip

Q: *Why three separate projections instead of using the raw vector three times?* → Because the roles differ. What a token wants to find (Q) is not the same as how it wants to be found (K), and neither is the same as the content it should hand over (V). Separate matrices let the model learn all three independently.

Self-Attention (2) : The Actual Maths

four steps, and you only need multiplication and softmax

$$\text{Attention}(Q, K, V) = \text{softmax}\left(Q \cdot K^T / \sqrt{d_k} \right) \cdot V$$

the single most quoted formula in modern AI ✍️

Step by step

1 Score : $Q \cdot K^T$

Dot every query with every key. A dot product is big when two vectors point the same way, so it is a ***similarity score***. For n tokens you get an n × n grid of scores.

2 Scale : $\div \sqrt{d_k}$

With d_k = 64 we divide by 8. Long vectors produce large dot products, and large numbers make softmax almost one-hot with near-zero gradients. Scaling keeps training healthy.

3 Softmax

Turns each row of scores into positive weights summing to 1. Now every token has a "budget of attention" of exactly 100% to spread around.

4 Weighted sum of V

Multiply each value vector by its weight and add them up. Output = the token's new, context-aware vector.

```
# shapes, n = tokens, d = model dim
Q, K, V : (n, d_k)
scores  : (n, n)      ← the n² cost
weights : (n, n)      ← each row sums to 1
output  : (n, d_v)
```

The cost you must remember

Every token compares itself to every other token, so work grows as $O(n^2)$ in sequence length. Double the prompt, quadruple the attention cost and memory.

Tokens	Score cells
1,000	1 million
10,000	100 million
100,000	10 billion

Worked mini example

Sentence: "***The cat sat, it was tired***". Query = the token "it". Scaled scores against each key:

Key token	Score	After softmax
"cat"	3.0	0.79
"sat"	1.0	0.11
"was"	0.5	0.06
"The"	0.0	0.04

New vector for "it" = $0.79 \cdot V_{cat} + 0.11 \cdot V_{sat} + 0.06 \cdot V_{was} + 0.04 \cdot V_{The}$

The word "it" now literally contains most of the meaning of "cat". Run this through 32 more layers and the model can track long chains of reference.

💡 Why softmax?

It is smooth (so it can be trained by gradients), always positive, and forces competition: giving more attention to one token means giving less to another.

⭐ Interview Tip

Two questions come up almost every time:

1. *Why divide by $\sqrt{d_k}$?* → to keep dot-product variance around 1 so softmax gradients do not collapse.
2. *What is the complexity of self-attention?* → $O(n^2 \cdot d)$ time and $O(n^2)$ memory for the score matrix.

Multi-Head Attention

why one attention is not enough

The idea

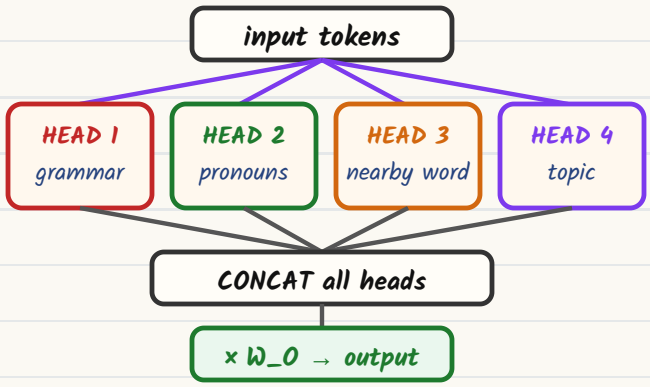
One attention pass can only ask one kind of question at a time. But a sentence has many relationships running at once: who is the subject, what does the pronoun refer to, which adjective belongs to which noun, what is the tense.

So we run attention ***many times in parallel*** with different W_Q, W_K, W_V. Each copy is a head. Then we stick the outputs side by side and pass them through one more matrix W_O to mix them back together.

```
# the split-and-rejoin trick
d_model = 512, heads = 8
  → each head works in 512/8 = 64 dims
head_i = Attention(Q WQi, K WKi, V WVi)
MHA = Concat(head_1 ... head_8) · WO
```

Total compute stays about the same as one big head, because the dimension is split, not duplicated. You get variety for free.

Picture it



Patterns researchers have actually found in heads

Head behaviour	What it does
Previous-token head	Almost always attends to the token immediately before
Syntactic head	Links verbs to their subjects and objects
Coreference head	Connects pronouns back to the noun they replace
Induction head	Spots a repeated pattern "A B ... A" and predicts B, the basis of in-context learning
Delimiter / sink head	Dumps unused attention on the first token or punctuation when nothing is relevant

Nobody assigns these jobs. They emerge from training, and a lot of interpretability research is about finding them.

Typical head counts

Model	Heads
Transformer base (2017)	8
BERT-base	12
GPT-3	96
Llama-3 8B	32 query heads, 8 KV heads

💡 Head dimension

$d_{head} = d_{model} \div$ number of heads. More heads means each one sees a narrower slice, so there is a balance to strike.

⭐ Interview Tip

Q: *Does multi-head attention cost 8× more than single-head?* → No. The model dimension is split across heads, so the total FLOPs are roughly unchanged. You gain multiple representation subspaces at almost no extra cost, which is why it is the default.

Masking : No Peeking at the Future

the trick that makes an LLM able to generate text at all

Why mask?

An LLM is trained to predict the next token. If a token could see the tokens after it, the answer would already be in the input and the model would learn nothing. That is cheating on the exam.

So before softmax we set every "future" score to **−infinity**. After softmax those weights become exactly 0. Position 3 can look at 0, 1, 2 and itself, and nothing beyond.

```
# causal mask applied to scores
scores = Q @ K.T / sqrt(d_k)
scores = scores.masked_fill(future, -inf)
weights = softmax(scores)   # future → 0
```

This is called causal or autoregressive masking. "Auto-regressive" simply means each new output is fed back in as input for the next step.

The triangle



Green = allowed to look. Red = blocked. Rows are queries, columns are keys.

Training vs generating

	Training	Generating (inference)
Input	Whole document at once	Prompt, then one token at a time
Predictions	All positions in parallel, n targets from one pass	One next token per pass
Mask	Essential, or it would copy the answer	Automatic, future tokens do not exist yet
Speed	Very parallel, GPU-friendly	Sequential, this is why generation feels slow

This parallel-training trick is called teacher forcing: the true previous tokens are fed in, so the model learns every position at the same time.

Masked ≠ Masked

- Causal masking (GPT): hide the future, predict the next token
- Masked language modelling (BERT): hide ~15% of random words anywhere and predict them, using both sides
- Padding mask: ignore filler tokens added to make a batch rectangular

⭐ Interview Tip

Q: *Why is inference slower than training per token?* → Training scores all positions in one parallel pass; generation must run a full forward pass for every single token, and each pass depends on the previous one.

The Feed-Forward Network (FFN)

the quiet half of the block, where most of the knowledge lives

What it does

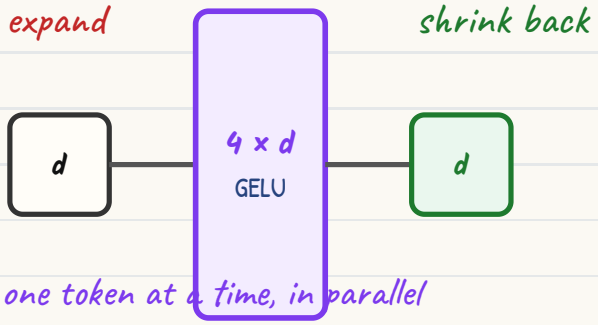
After attention has gathered context, each token goes through a small two-layer network **on its own**. No mixing with neighbours here. Same weights applied to every position.

```
# classic FFN (2017)
h = x @ W1 + b1      # expand: d → 4d
h = GELU(h)           # non-linearity
y = h @ W2 + b2      # shrink: 4d → d

# modern SwiGLU version (Llama, PaLM)
y = (SiLU(x @ W_gate) * (x @ W_up)) @ W_down
```

Expand → activate → shrink. Widening to 4× gives the layer room to separate patterns; the non-linearity is what stops the whole network from collapsing into one big matrix multiply.

Shape of it



Activation	Used by
ReLU	Original 2017 Transformer
GELU	BERT, GPT-2, GPT-3
SwiGLU	Llama, Mistral, PaLM, most 2023+ models

Where do facts live? 🧠

Research treats the FFN as a key-value memory. The first matrix detects a pattern ("this looks like the subject is a country capital question"), the second writes out the associated information. Editing individual facts in a model, work like ROME and MEMIT, targets these FFN weights.

A rough division of labour:

- Attention = routing and reference, "which words matter here"
- FFN = recall and transformation, "what do I know about this"

Parameter share per block

Part	Params (d = d_model)	Share
Attention (W_Q, W_K, W_V, W_O)	$4d^2$	≈ 33%
FFN (W1, W2 at 4×)	$8d^2$	≈ 67%
Total per block	$\approx 12d^2$	100%

Two thirds of an LLM's weights sit in the feed-forward layers, even though attention gets all the attention.

💡 Quick maths

Llama-3 8B: d = 4096, 32 layers. $12 \times 4096^2 \times 32 \approx 6.4$ billion, close to the real 8B once embeddings and the wider SwiGLU are counted. You can sanity-check any model this way.

⭐ Interview Tip

Q: *Why do we need the FFN if attention already mixes information?* → Attention is a weighted average, which is a linear operation over values. Without a per-token non-linear layer the stack would have limited expressive power and could not store learned knowledge.

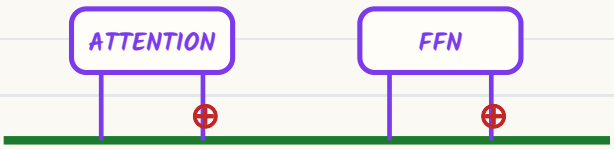
Residuals, Normalisation & Depth

the plumbing that lets you stack 100 layers without the thing falling apart

Residual connections

Every sub-layer computes **output = x + f(x)**, not just f(x). Two reasons this matters:

- ✓ Easy default: if a layer has nothing useful to add, it can output near zero and the signal passes through untouched
- ✓ Gradient highway: during training the gradient flows back through the "+ x" path without shrinking, which kills the vanishing-gradient problem



the residual stream

Normalisation

LayerNorm takes one token's vector, subtracts its mean, divides by its standard deviation, then applies learned scale and shift. It does this per token, never across the batch, so batch size and sequence length do not matter.

$$y = \gamma \cdot (x - \text{mean}) / \sqrt{(\text{var} + \varepsilon)} + \beta$$

RMSNorm is the modern simplification: skip the mean subtraction and the shift, divide by the root-mean-square only. Fewer operations, same stability, used by Llama and most recent models.

$$y = \gamma \cdot x / \sqrt{(\text{mean}(x^2) + \varepsilon)}$$

Why bother: as vectors pass through dozens of layers their scale drifts. Normalising each time keeps every layer receiving inputs in a familiar range.

Norm comparison

Type	Normalises over	Good for
BatchNorm	The batch, per feature	CNNs; poor fit for variable-length text
LayerNorm	Features of one token	The Transformer standard
RMSNorm	Same, without centring	Faster; Llama, Mistral, Gemma

Placement matters: pre-norm (norm → sub-layer → add) is the modern default because the residual path stays clean all the way from input to output. Post-norm, as in the original paper, needs a learning-rate warm-up and gets fragile past ~20 layers.

Why go deep at all?

- Each block can only do a limited amount of routing plus one recall step
- Multi-step reasoning needs several rounds: find the subject → recall its property → compare → decide
- Depth also buys abstraction: characters → words → phrases → intent

💡 Sanity check

Remove residual connections from a 32-layer Transformer and it will barely train. Remove normalisation and loss will spike to NaN. Neither is decoration.

⭐ Interview Tip

Q: *Why LayerNorm and not BatchNorm in NLP?* → Sentences vary in length and batches are small at inference, so batch statistics are unstable and would leak information across examples. LayerNorm depends only on the single token being processed, so it behaves identically in training and serving.

Encoder, Decoder, or Both?

three family trees from the same 2017 blueprint

ENCODER-ONLY

reads whole text both directions 👀

BERT, RoBERTa, DeBERTa

Trained by hiding random words and guessing them. Output is a rich vector per token, not new text.

Use for: classification, sentiment, NER, search embeddings, ranking.

DECODER-ONLY ⭐

reads left to right writes next token

GPT, Claude, Llama, Mistral, Gemma

Trained purely on next-token prediction with a causal mask. Every task becomes "continue this text".

Use for: chat, code, agents, reasoning, basically everything today.

ENCODER-DECODER

read input fully → then write output

T5, BART, original 2017 model

Encoder digests the source, decoder writes the target and peeks at the encoder via cross-attention.

Use for: translation, summarisation, speech-to-text.

Side by side

	Encoder-only	Decoder-only	Encoder-Decoder
Attention	Bidirectional	Causal (masked)	Bi in encoder, causal + cross in decoder
Training goal	Fill in masked words	Predict next token	Reconstruct target from source
Can generate?	No	Yes	Yes
Best at	Understanding	Generating + everything via prompting	Mapping one sequence to another
Example size	110M (BERT-base)	8B → 400B+	220M → 11B (T5)

Why decoder-only won

- ✓ One simple objective works for every task
- ✓ Training data is just raw text, no labels needed
- ✓ Scales cleanly and predictably
- ✓ Prompting replaces task-specific heads
- ✓ KV caching makes generation efficient

Encoders are far from dead though: your RAG pipeline's embedding model and reranker are usually encoder-based, because they are small, fast and bidirectional.

💡 Cross-attention in one line

Queries come from the text being written, Keys and Values come from the text being read. It is the bridge between two sequences, and it is what image-to-text and speech models still use heavily.

⭐ Interview Tip

Q: *Would you use BERT or GPT for classifying support tickets?* → A fine-tuned encoder is usually cheaper, faster and more accurate for a fixed label set. Reach for an LLM when labels change often, examples are scarce, or you need an explanation with the label.

Output : Logits → Softmax → Word

how the model finally decides what to say

The last stretch

1 Take the last token's vector

After the final block, only the vector at the last position matters for the next word.

2 LM head (unembedding)

Multiply by a matrix of size d × vocab. Result: one raw score, a logit, for every token in the vocabulary. Often this matrix is tied to the input embedding matrix to save memory.

3 Softmax

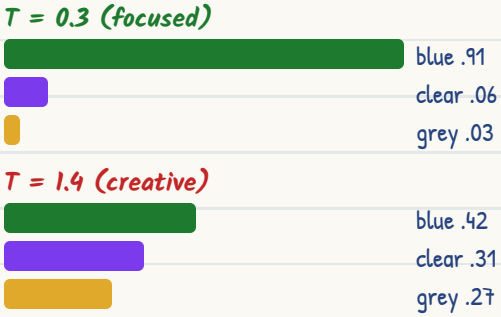
Logits → probabilities that sum to 1 across all 128,000 candidates.

4 Pick one

Greedy, or sample using temperature / top-k / top-p.

```
# temperature is applied BEFORE softmax
p = softmax(logits / T)
T = 0.0 → always the top token
T = 0.7 → balanced, common default
T = 1.5 → wild, creative, error-prone
```

Same logits, different T



Low temperature sharpens the peak, high temperature flattens it. The model's knowledge never changes, only how boldly you sample from it.

💡 Why answers differ each time

Nothing random happens inside the network. Randomness enters only at this sampling step. Set temperature to 0 and you get near-deterministic output.

Decoding strategies you will see in every API

Setting	What it does	Good default
Greedy	Always take the highest-probability token	Extraction, classification, JSON output
Temperature	Flattens or sharpens the whole distribution	0 for facts, 0.7 for chat, 1.0+ for brainstorming
Top-k	Keep only the k most likely tokens, then sample	k = 40
Top-p (nucleus)	Keep the smallest set of tokens whose probabilities add to p	p = 0.9, adapts to how confident the model is
Repetition / frequency penalty	Lowers scores of tokens already used	Small values, stops loops
Beam search	Explores several candidate sequences and keeps the best overall	Translation; too bland for open chat

Then it loops 🔁

The chosen token is appended to the input and the entire forward pass runs again. A 500-token answer means 500 full passes through every layer. This is why output length drives cost and latency far more than input length.

⭐ Interview Tip

Q: *Difference between top-k and top-p?* → Top-k always keeps a fixed number of candidates regardless of confidence. Top-p keeps a variable number based on cumulative probability, so it narrows when the model is sure and widens when it is not.

How an LLM is Trained

from raw internet text to something that follows instructions

Stage 1 · Pre-training

Feed trillions of tokens of text. For every position, the model predicts the next token, and we compare its guess with the truth using cross-entropy loss.

```
# loss for one position
loss = -log( p[correct_token] )

p = 0.9  → loss 0.11   # confident, right
p = 0.1  → loss 2.30   # wrong, punished
```

Backpropagation then works out how each of the billions of weights contributed to the error, and an optimiser (usually AdamW) nudges every weight a tiny step in the better direction. Repeat for months on thousands of GPUs.

No human labels anywhere, the text is its own answer key, which is why this is called self-supervised learning.

Perplexity = exp(average loss). Read it as "on average, how many tokens is the model torn between". Lower is better.

The three stages

1. PRE-TRAINING

trillions of tokens · months · $$$

result: a good text predictor

↓

2. SUPERVISED FINE-TUNING (SFT)

thousands of human-written

instruction → answer pairs

result: it follows instructions

↓

3. PREFERENCE TUNING

RLHF or DPO · humans rank

answer A vs answer B

result: helpful, safer, better style

↓

the assistant you talk to 🤖

A base model only completes text, it will happily continue your question with ten more questions. Stages 2 and 3 turn it into something that answers.

Words to know

Term	Plain meaning
Epoch	One full pass over the training data
Batch	How many sequences are processed together before updating weights
Learning rate	Step size. Too big and it diverges, too small and it crawls
Overfitting	Memorising the training set instead of learning patterns
Scaling laws	Loss falls predictably as model size, data and compute grow together

Cheaper ways to adapt a model

- Prompting: no training at all, just better instructions and examples
- RAG: retrieve your documents into the prompt. Best for facts that change
- LoRA / QLoRA: freeze the big model, train small adapters. Millions of params, not billions
- Full fine-tuning: retrain all weights. Expensive, rarely the first answer

💡 Practical rule

Fine-tuning teaches behaviour and format. RAG supplies knowledge. Mixing these up is the single most common mistake in LLM projects.

⭐ Interview Tip

Q: *My model gives outdated answers. Fine-tune or RAG?* → RAG. Knowledge that changes should live in a retrievable store, not in frozen weights. Fine-tune only for tone, structure or a domain skill prompting cannot reach.

Context Window & the KV Cache

why long chats get slow, expensive, and forgetful

Context window

The maximum number of tokens the model can hold in one pass: your system prompt + chat history + documents + its own answer. Older models had 2K, today 128K to 1M+ is common.

Everything outside that window simply does not exist for the model. A chatbot "remembering" your earlier messages just means your app is resending them every single turn.

LOST IN THE MIDDLE 📉

Models attend most reliably to the start and the end of a long context. Information buried in the middle is more likely to be missed, so put critical instructions at the top and the actual question at the bottom.

The KV cache

Generating token 501 would normally recompute Keys and Values for all 500 earlier tokens. Wasteful, because those never change. So we store them once and reuse them.



only the new token is computed →

$O(n^2)$ per answer becomes $O(n)$ per step

Cost: memory. Cache size ≈ 2 × layers × kv_heads × head_dim × tokens × bytes. For long contexts this can exceed the size of the model weights themselves.

Two phases of inference

Phase	What happens	Bound by
Prefill	Whole prompt processed in one parallel pass, cache filled	Compute. Sets "time to first token"
Decode	One token per pass, reading the cache	Memory bandwidth. Sets tokens per second

This is why input tokens are usually billed cheaper than output tokens, and why prompt caching gives such a big speed-up on repeated system prompts.

Tricks that make long context possible

- FlashAttention: same maths, but computed in tiles inside fast GPU memory, so the n × n matrix is never fully written out
- MQA / GQA: many query heads share fewer key-value heads, shrinking the cache several times over
- Sliding window: each token only attends to the last k tokens
- Quantised cache: store K and V in 8-bit or 4-bit
- PagedAttention (vLLM): manage cache in pages like virtual memory, far less waste

💡 Cost lever

Long context is not free intelligence. Retrieving the right 2,000 tokens usually beats dumping 100,000 and hoping. Cheaper, faster and often more accurate.

⭐ Interview Tip

Q: *Does the KV cache change the model's answers?* → No. It is a pure engineering optimisation that stores results already computed. Same output, far less repeated work.

Modern Upgrades & Honest Limits

what changed since 2017, and what still goes wrong

What a 2025-era LLM changed from the original paper

Upgrade	Replaces	Why
RoPE	Sinusoidal position encoding	Relative positions, extends to longer context
RMSNorm, pre-norm	LayerNorm, post-norm	Faster and stable at 80+ layers
SwiGLU	ReLU FFN	Better quality at equal parameter count
GQA / MQA	Full multi-head attention	Much smaller KV cache, faster serving
Mixture of Experts	One dense FFN per layer	Many expert FFNs, only 1–2 run per token
FlashAttention	Naive attention kernel	Same result, far less memory traffic
Quantisation (INT8/4)	FP32 / FP16 weights	Runs a 70B model on one GPU

DO ✓

- ✓ Count tokens, not words, when budgeting cost
- ✓ Put instructions at the start, the question at the end
- ✓ Use temperature 0 for anything that must be exact
- ✓ Ground factual answers with retrieval
- ✓ Ask for a fixed schema when you need parseable output
- ✓ Cache the stable part of your prompt
- ✓ Evaluate on a real test set, not vibes

AVOID ✗

- ✗ Treating the model as a database of current facts
- ✗ Fine-tuning to inject knowledge that changes weekly
- ✗ Dumping 100K tokens when 2K would do
- ✗ Trusting arithmetic, dates or citations without a check
- ✗ Sending personal or confidential data without a policy

Known weaknesses, and why they happen

Problem	Root cause
Hallucination	It optimises for plausible tokens, not true ones
No memory	Weights are frozen; your app resends the history
Weak arithmetic	Digits tokenise oddly, no calculator inside
Knowledge cutoff	Nothing after the training date is in the weights
Prompt injection	Instructions and data share one channel
Bias	Training-data patterns get reproduced, even amplified

💡 The fix is architecture, not scolding

Give the model tools: a calculator for maths, search for facts, code execution for logic, a validator for schemas. Prompts alone will not repair a structural limit.

⭐ Interview Tip

Q: *Why do LLMs hallucinate?* → The objective rewards likely continuations, not verified ones, and nothing in the loss teaches it to say "I don't know". Grounding and tools reduce it, they do not remove it.

Rapid-Fire Interview Q & A

15 questions that come up again and again

1 What is a Transformer?

A neural network built from stacked blocks of self-attention plus a feed-forward layer, with residuals and normalisation, processing all tokens in parallel.

2 What is self-attention?

Each token builds a query, compares it to every token's key, and takes a weighted average of their values.

3 Why divide by $\sqrt{d_k}$?

Keeps dot-product magnitudes stable so softmax does not saturate and kill the gradients.

4 Why multiple heads?

Different heads capture different relationships in parallel, at roughly the same total cost.

5 Purpose of positional encoding?

Attention is order-blind; positional information restores word order.

6 Why residual connections?

They let gradients flow and let each layer learn a small correction instead of the whole mapping.

7 LayerNorm vs BatchNorm?

LayerNorm normalises within one token, so it is independent of batch size and sequence length.

8 Encoder vs decoder?

Encoder sees both directions and understands; decoder is causally masked and generates.

9 Why a causal mask?

So a position cannot see the answer it is being trained to predict.

10 Complexity of attention?

$O(n^2 \cdot d)$ time, $O(n^2)$ memory in sequence length n. The main reason long context is expensive.

11 What does temperature do?

Divides the logits before softmax. Lower means sharper and more deterministic, higher means flatter and more varied.

12 What is the KV cache?

Stored keys and values from earlier tokens, reused so each new token needs only one fresh computation.

13 Where are facts stored?

Mostly in the feed-forward layers, which hold about two thirds of the parameters and act like a key-value memory.

14 RAG vs fine-tuning?

RAG supplies knowledge at query time; fine-tuning shapes behaviour, tone and format.

15 What is in-context learning?

The model picks up a pattern from examples in the prompt and continues it, with no weight updates at all.

💡 How to answer well

Give the one-line definition first, then one sentence of "why it exists". Interviewers are checking whether you understand the motivation, not whether you memorised a formula.

⭐ The question that separates candidates

"Walk me through what happens when I type 'hello' into ChatGPT." → Tokenize → embed → add position → run through N blocks where attention mixes tokens and the FFN processes each one → final norm → LM head gives a logit per vocabulary token → softmax → sample with temperature and top-p → append the token → repeat until a stop token. Mention the KV cache and you have covered inference too.

Summary & 30-Day Roadmap

The whole thing in one line

TOKENS → EMBEDDINGS + POSITION → ATTENTION + FFN × N → LOGITS + SOFTMAX → NEXT TOKEN ↺

Glossary you should be able to explain

Term	One-line meaning
Token	Smallest chunk of text the model sees
Embedding	Vector that carries a token's meaning
Q / K / V	What I want / what I offer / what I give
Head	One independent attention pattern
Logit	Raw score before softmax
Softmax	Scores → probabilities summing to 1
Autoregressive	Output is fed back in as the next input
Context window	Max tokens visible in one pass
Parameters	The learned numbers, the model's "weights"
Inference	Using a trained model to produce output
Hallucination	A fluent answer that happens to be false
RAG	Fetch documents, put them in the prompt

30-day plan

WEEK 1 · Foundations
Neural network basics, matrix multiplication, softmax. Play with a tokenizer and watch your own sentences split.

WEEK 2 · Attention
Read "Attention Is All You Need" and The Illustrated Transformer. Implement single-head attention in NumPy by hand.

WEEK 3 · Build one
Code a tiny GPT from scratch and train it on a small text file. Then fine-tune a small open model with LoRA.

WEEK 4 · Ship something
Build a RAG app end to end: chunk → embed → vector store → retrieve → prompt → evaluate. Measure cost and latency.

💡 Key Takeaway

A Transformer has two moves: tokens share information (attention) and each token thinks alone (FFN). Stack them deep, train on enough text, and that is an LLM. RoPE, GQA, MoE and quantisation are engineering around those two moves.

⭐ Final Interview Tip

If you can draw the block diagram on a whiteboard and explain why each arrow exists, you are ahead of most candidates. Practise the 30-second pipeline out loud until it is boring.

Myths, cleared up

People say	What is actually true
"It searches the internet"	Not by default. It generates from weights unless a search tool is wired in
"Bigger context = smarter"	More room to read, not more reasoning. Relevance beats volume
"It remembers our last chat"	Only if the app resends that text or stores it for you

YOU NOW KNOW HOW LLMS WORK! 🎉 save & revise

👤➕ FOLLOW | 📤 SHARE | 💬 COMMENT | ❤️ LIKE